

Módulo de Encuestas de Seguimiento BETA

Cómo funciona: del ícono en la tabla, al archivo guardado en disco

1. Qué tiene de distinto este módulo

Todo lo que armamos antes (Obras, Acciones, Origen de Inversión, COCI) sigue exactamente el mismo patrón: el navegador manda **texto/números como JSON**, Laravel los guarda en MySQL, y regresa JSON de vuelta. Encuestas rompe ese patrón en un punto clave: además de texto, también sube **archivos** (fotos y documentos) — y los archivos no pueden viajar dentro de un JSON. Por eso este módulo usa un mecanismo distinto llamado `FormData`, y vive en su propio archivo `Encuestas.js`, separado de `Obras.js`, para no arriesgar nada de lo que ya funciona.

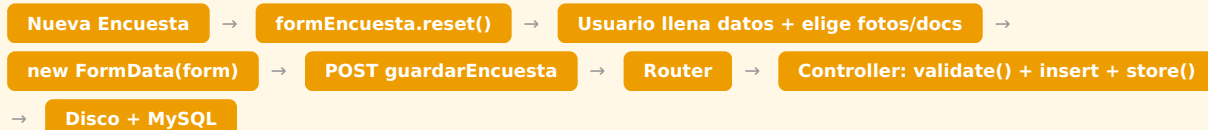
2. El camino completo: abrir la bitácora



El ícono `fa-clipboard-check` que ves en cada fila de la tabla de Obras (donde antes estaba el de Localidades) llama a `EncuestasModule.verEncuestas(idobra)`. Esto abre el modal y dispara **dos peticiones GET en paralelo**: una trae el catálogo de Estatus para el combo, y otra trae la bitácora completa de esa obra (todas sus encuestas, ordenadas de la más reciente a la más antigua).

Del lado de Laravel, `getEncuestas($idobra)` hace algo interesante que vale la pena señalar: además del `SELECT` principal, recorre cada fila y le agrega **cuántas fotos y documentos tiene** (dos consultas extra por fila, a las tablas `TblD_EncuestaFotos` y `TblD_EncuestaDocumentos`). Así el listado puede mostrar "4 fotos · 3 docs" sin tener que cargar el detalle completo de cada encuesta.

3. El camino completo: guardar una encuesta nueva (con archivos)



3.1 Por qué FormData y no JSON

En todos los módulos anteriores, `guardarObra()` / `guardarAccion()` armaban un objeto JavaScript y lo mandaban así:

```
data: JSON.stringify(payload),
contentType: 'application/json'
```

Un archivo binario (una foto, un PDF) no se puede meter dentro de un string JSON. La forma estándar de mandar archivos por HTTP es `multipart/form-data`, y en JavaScript eso se arma con el objeto nativo `FormData`:

```
var formData = new FormData(document.getElementById('formEncuesta'));
formData.append('idobra', idObraActual);
```

```

formData.append('fechaVisita', $('#txtFechaVisita').val());
// ...

$.ajax({
    url: API + 'guardarEncuesta',
    method: 'POST',
    data: formData,
    processData: false, // no dejar que jQuery lo convierta
    contentType: false // dejar que el navegador arme el header correcto
});

```

Los dos flags `processData: false` y `contentType: false` son obligatorios cuando se manda un `FormData` — sin ellos, jQuery intenta "ayudar" convirtiendo los datos a texto plano, y los archivos se pierden en el camino.

3.2 Cómo entran los archivos en el HTML

El atributo clave está en dos lugares del formulario:

```

<form id="formEncuesta" enctype="multipart/form-data">
    ...
    <input type="file" name="fotos[]" accept="image/*" multiple>
    <input type="file" name="documentos[]" multiple>
</form>

```

- `enctype="multipart/form-data"` en el `<form>` — sin esto, Laravel nunca recibe los archivos, aunque el JS los mande bien.
- El `[]` al final de `name="fotos[]"` es lo que le dice al navegador "puede haber varios archivos con este mismo nombre" — así `$request->file('fotos')` del lado de Laravel regresa un arreglo, no un solo archivo.
- `multiple` es lo que permite seleccionar más de un archivo a la vez desde el explorador de Windows.

3.3 Qué hace el Controller con esos archivos

```

$request->validate([
    'fotos.*' => 'nullable|image|max:8192', // 8 MB por foto
    'documentos.*' => 'nullable|file|max:15360', // 15 MB por documento
]);

if ($request->hasFile('fotos')) {
    foreach ($request->file('fotos') as $foto) {
        $ruta = $foto->store('encuestas/fotos', 'uploads');
        DB::table('TblD_EncuestaFotos')->insert([
            'IDEncuesta' => $idEncuesta,
            'RutaArchivo' => $ruta,
            'NombreOriginal' => $foto->getClientOriginalName(),
            'FechaSubida' => now(),
        ]);
    }
}

```

Por cada foto: Laravel la guarda físicamente en el disco (le pone un nombre único automático para que nunca choque con otro archivo, aunque el usuario suba dos fotos llamadas `IMG_001.jpg`), y guarda en la tabla `TblD_EncuestaFotos` tanto la ruta real como el nombre original (para mostrárselo bonito al usuario después). Documentos funciona exactamente igual, en su propia tabla.

¿Por qué "uploads" y no el disco "public" de Laravel?

El disco por default de Laravel (`storage/app/public`) requiere correr `php artisan storage:link` en cada servidor donde se instale el proyecto — y algunos hostings compartidos ni siquiera permiten crear ese enlace simbólico por seguridad. Se

configuró un disco personalizado `uploads` que guarda directo dentro de `public/uploads/`, así los archivos son accesibles sin ningún paso de configuración extra, en cualquier servidor.

4. El camino completo: editar una encuesta existente

Clic en la bitácora → `form.reset()` → `GET getEncuestaById/{id}` → Precarga campos + miniaturas
→ Guardar (mismo `guardarEncuesta`)

Lección aprendida (bug real que corregimos): al editar, si no se limpia el `<input type="file">` antes de precargar los demás campos, el navegador conserva el archivo seleccionado de una edición anterior. Al guardar, ese archivo "fantasma" se vuelve a subir, duplicando el último documento. La corrección fue agregar `$('#formEncuesta')[0].reset()` al inicio de `editarEncuesta()`, antes de precargar los valores reales.

5. Borrar fotos/documentos individuales

Cada miniatura de foto y cada renglón de documento en el modo edición trae su propio botón de basura, que llama a `eliminarFotoExistente(idFoto)` o `eliminarDocumentoExistente(idDocumento)` — estos mandan un `DELETE` directo a Laravel, que borra **tanto el registro en la base de datos como el archivo físico del disco** (con `Storage::disk('uploads')->delete(...)`), para no dejar basura huérfana acumulándose en el servidor con el tiempo.

6. Los archivos del módulo, de un vistazo

```
app/Http/Controllers/EncuestasController.php ← toda la logica (Controller nuevo)
config/filesystems.php                       ← disco "uploads" (sin symlink)
routes/web.php                               ← 7 rutas nuevas, prefix api/Encuestas
public/assets/js/Encuestas.js                 ← JS nuevo, separado de Obras.js
public/uploads/encuestas/fotos/               ← donde llegan las fotos físicamente
public/uploads/encuestas/documentos/          ← donde llegan los documentos

Tablas nuevas en MySQL:
TblC_EstatusEncuesta ← catalogo (En tiempo / Retrasada / Detenida / Concluida)
TblD_Encuestas       ← una fila por visita de seguimiento
TblD_EncuestaFotos   ← una fila POR FOTO (relacion 1 a muchos)
TblD_EncuestaDocumentos ← una fila POR DOCUMENTO (relacion 1 a muchos)
```

7. Comparación rápida con los módulos anteriores

Concepto	Obras / Acciones / Origen	Encuestas
¿Existe en el sistema C# original?	Sí, se migró 1:1	No, es funcionalidad nueva (beta)
Formato de envío	JSON	FormData (multipart)
Archivo JS	Obras.js	Encuestas.js (separado)
Relación con archivos	Ninguna	1 encuesta → muchas fotos/documentos
Configuración extra requerida	Ninguna	Disco "uploads" en filesystems.php

8. Lo que sigue (pendiente)

- Definir con más precisión el requerimiento real de negocio (por ahora el diseño de campos fue una propuesta razonable, no un requerimiento formal).
- Si se necesita que las fotos/documentos sean privados (solo visibles con sesión iniciada), hay que migrar del disco público directo a una ruta de Laravel que valide sesión antes de entregar el archivo.
- Posible reporte/exportación de la bitácora completa de una obra a PDF o Excel.